

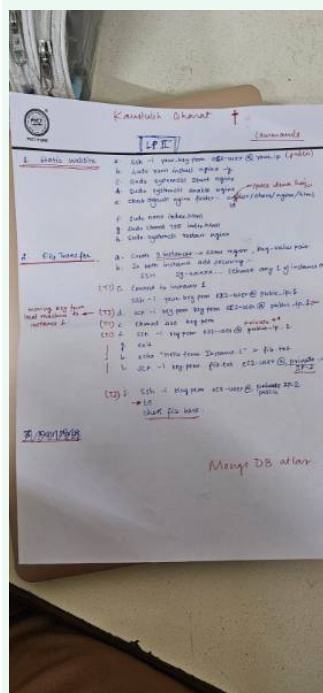
COMMANDS EXPLANATION

FIELD LABEL	FIELD NAME	DATA TYPE
Created By	CreatedById	Lookup(User)
EXPIRYDATE	EXPIRYDATE_c	Date
Last Modified By	LastModifiedById	Lookup(User)
MANUFACTUREDATE	MANUFACTUREDATE_c	Date
Owner	OwnerId	Lookup(User,Group)
PRODUCTINVEN/FORMGMT Name	Name	Text(80)
SERIALNO	SERIALNO_c	Number(18, 0) (Unique)

Pasted text(19).txt

Document

GIVE CODE FOR THIS



GIVE ME IN TEXT FROMAT THESE STEPS

LP II Commands

1. Static Website Hosting using NGINX

a. Connect to EC2 Instance

Bash

```
ssh -i your-key.pem ec2-user@your-public-ip
```

b. Install NGINX

Bash
`sudo yum install nginx -y`

c. Start NGINX Service

Bash
`sudo systemctl start nginx`

d. Enable NGINX

Bash
`sudo systemctl enable nginx`

e. Check Default NGINX Folder

Bash
`cd /usr/share/nginx/html`

f. Create/Edit index.html

Bash
`sudo nano index.html`

g. Give Permissions

Bash
`sudo chmod 755 index.html`

h. Restart NGINX

Bash
`sudo systemctl restart nginx`

2. File Transfer Between Two EC2 Instances

Step A — Create Instances

- Create 2 EC2 instances
 - Both should be in the same region
 - Use same key-value pair
-

Step B — Add Security Rule

In both instances add security rule:

- Type: SSH

- Source: Security Group of other instance

Example:

sg-xxxxxx

Terminal 1 (T1)

c. Connect to Instance 1

Bash

```
ssh -i your-key.pem ec2-user@public-ip-1
```

d. Copy Key to Instance 1

Bash

```
scp -i key.pem key.pem ec2-user@public-ip-1:~
```

e. Change Permission of Key

Bash

```
chmod 400 key.pem
```

f. Connect to Instance 2 from Instance 1

Bash

```
ssh -i key.pem ec2-user@private-ip-2
```

g. Exit

Bash

```
exit
```

h. Create File

Bash

```
echo "Hello from Instance-1" > file.txt
```

i. Transfer File to Instance 2

Bash

```
scp -i key.pem file.txt ec2-user@private-ip-2:~
```

Terminal 2 (T2)

j. Connect to Instance 2

Bash

```
ssh -i key.pem ec2-user@public-ip-2
```

k. Check File

Bash

```
ls
```

Check whether `file.txt` is present.

MongoDB Atlas

(Only title written in image, no steps provided.)

IN ONE GO GIVE COMMANDS

Bash

```
# =====
# 1. STATIC WEBSITE USING NGINX
# =====
```

```
ssh -i your-key.pem ec2-user@your-public-ip
```

```
sudo yum install nginx -y
```

```
sudo systemctl start nginx
```

```
sudo systemctl enable nginx
```

```
cd /usr/share/nginx/html
```

```
sudo nano index.html
```

```
sudo chmod 755 index.html
```

```
sudo systemctl restart nginx
```

```
# =====
# 2. FILE TRANSFER BETWEEN TWO EC2 INSTANCES
# =====
```

```
# (T1) Connect to Instance-1
```

```
ssh -i your-key.pem ec2-user@public-ip-1
```

```
# Copy key to Instance-1
```

```
scp -i key.pem key.pem ec2-user@public-ip-1:~
```

```
# Give permission to key
```

```
chmod 400 key.pem
```

```
# Connect to Instance-2 from Instance-1
ssh -i key.pem ec2-user@private-ip-2

# Exit from Instance-2
exit

# Create file in Instance-1
echo "Hello from Instance-1" > file.txt

# Transfer file to Instance-2
scp -i key.pem file.txt ec2-user@private-ip-2:~

# (T2) Connect to Instance-2
ssh -i key.pem ec2-user@public-ip-2

# Check transferred file
ls
```

NOW TELL ME EXACTLY WHAT THESE LINES DO

1. Static Website Hosting Commands

Connect to EC2 Instance

Bash

```
ssh -i your-key.pem ec2-user@your-public-ip
```

What it does:

- `ssh` → Secure Shell login
- `-i your-key.pem` → Uses your PEM key for authentication
- `ec2-user` → Default AWS Linux username
- `your-public-ip` → Public IP of EC2 instance

This command connects your local machine to the EC2 server.

Install NGINX

Bash

```
sudo yum install nginx -y
```

What it does:

- **sudo** → Run command as administrator
- **yum** → Package manager in Amazon Linux
- **install nginx** → Downloads and installs NGINX web server
- **-y** → Automatically says “yes” to prompts

This installs the web server software.

Start NGINX Service

Bash

```
sudo systemctl start nginx
```

What it does:

- **systemctl** → Controls Linux services
- **start nginx** → Starts the NGINX service

Without this, the website won't run.

Enable NGINX

Bash

```
sudo systemctl enable nginx
```

What it does:

Makes NGINX automatically start whenever the server reboots.

Move to NGINX Website Folder

Bash

```
cd /usr/share/nginx/html
```

What it does:

- **cd** → Change directory
- **/usr/share/nginx/html** → Default folder from where NGINX serves webpages

Anything inside this folder becomes your website content.

Edit index.html

Bash

```
sudo nano index.html
```

What it does:

- `nano` → Terminal text editor
- Opens `index.html`

You write your HTML webpage here.

Change File Permission

Bash

```
sudo chmod 755 index.html
```

What it does:

- `chmod` → Change file permissions
- 755 means:
 - Owner → read/write/execute
 - Others → read/execute

Allows NGINX to access the webpage file properly.

Restart NGINX

Bash

```
sudo systemctl restart nginx
```

What it does:

Restarts NGINX so latest webpage changes become visible.

2. File Transfer Between EC2 Instances

Connect to Instance-1

Bash

```
ssh -i your-key.pem ec2-user@public-ip-1
```

What it does:

Connects to the first EC2 instance.

Copy PEM Key to Instance-1

Bash

```
scp -i key.pem key.pem ec2-user@public-ip-1:~
```

What it does:

Printed using [ChatGPT to PDF](#), powered by PDFCrowd [HTML to PDF API](#).

- `scp` → Secure Copy command
- Copies `key.pem` from local machine to Instance-1

Why?

Because Instance-1 needs the key to connect to Instance-2.

Change Key Permission

Bash
`chmod 400 key.pem`

What it does:

Makes the PEM key secure.

Permission `400` means:

- Owner can only read
- Nobody else can access

SSH requires secure permissions for keys.

Connect Instance-1 to Instance-2

Bash
`ssh -i key.pem ec2-user@private-ip-2`

What it does:

From Instance-1, connects to Instance-2 using private IP.

Private IP works because both instances are in same VPC/network.

Exit Connection

Bash
`exit`

What it does:

Logs out from current SSH session.

Create a File

Bash
`echo "Hello from Instance-1" > file.txt`

What it does:

- `echo` → Prints text
- `>` → Redirects output into file
- Creates `file.txt`

Contents:

Hello from Instance-1

Transfer File to Instance-2

Bash

```
scp -i key.pem file.txt ec2-user@private-ip-2:~
```

What it does:

Copies `file.txt` from Instance-1 to Instance-2.

Connect to Instance-2

Bash

```
ssh -i key.pem ec2-user@public-ip-2
```

What it does:

Directly logs into Instance-2.

Check File

Bash

```
ls
```

What it does:

Lists all files in current directory.

You verify whether `file.txt` arrived successfully.

WHAT IS NGINX

NGINX

Definition

NGINX (pronounced “**Engine-X**”) is a **web server software** used to:

- Host websites

- Serve web pages
- Handle client requests
- Work as a reverse proxy
- Load balance traffic

It is one of the most popular web servers in the world.

Simple Understanding

When someone opens your website in a browser:

Browser → NGINX Server → Your Website Files

NGINX receives the request and sends back the webpage.

Example

Suppose your EC2 server contains:

`/usr/share/nginx/html/index.html`

When a user opens:

`http://your-public-ip`

NGINX reads `index.html` and displays the webpage in browser.

Why We Install NGINX

Without NGINX:

- Your HTML file just sits inside the server
- Nobody can access it through browser

With NGINX:

- Your server becomes a web server
 - Website becomes accessible on internet
-

Main Uses of NGINX

1. Static Website Hosting

Hosts:

- HTML

- CSS
- JavaScript files

Example:

Portfolio Website
College Project
Landing Page

2. Reverse Proxy

NGINX sits in front of backend servers.

Client → NGINX → Node.js / Django / Spring Boot

It forwards requests to backend applications.

3. Load Balancing

If many users visit website:

NGINX distributes traffic across multiple servers

This prevents server overload.

4. Security

Can:

- Hide backend server
 - Filter requests
 - Handle SSL/HTTPS
-

Why NGINX is Popular

- Very fast
 - Lightweight
 - Handles many users
 - Uses less memory
 - Stable and reliable
-

In Your Practical

You used:

Bash

```
sudo yum install nginx -y
```

to install NGINX.

Then:

Bash

```
sudo systemctl start nginx
```

started the web server.

After putting `index.html` in:

```
/usr/share/nginx/html
```

your webpage became live on the EC2 public IP.

One-Line Viva Answer

NGINX is an open-source web server used to host websites, handle client requests, reverse proxy, and load balance traffic efficiently.

```
// sudo dnf update -y
// curl -fsSL https://rpm.nodesource.com/setup_18.x | sudo bash -
// sudo dnf install -y nodejs git
EXPLAIN THESE COMMADNS ASLO
```

1. Update System Packages

Bash

```
sudo dnf update -y
```

What it does

- **sudo**
→ Runs command as administrator/root user
- **dnf**
→ Package manager in newer Linux systems like Amazon Linux 2023
- **update**
→ Updates all installed software/packages to latest versions
- **-y**
→ Automatically answers “yes” to confirmation prompts

Simple Meaning

This command updates your Linux system and installed packages.

Why We Use It

Before installing software, we update system packages to:

- Fix bugs
 - Improve security
 - Avoid installation issues
-

2. Add Node.js Repository

Bash

```
curl -fsSL https://rpm.nodesource.com/setup_18.x | sudo bash -
```

What it does

This command downloads and runs the NodeSource setup script for Node.js 18.

Breaking it Down

curl

Used to download data/files from internet.

-f

Fails silently if server error occurs.

-s

Silent mode (less output).

-S

Shows errors if something fails.

-L

Follows redirects automatically.

URL

`https://rpm.nodesource.com/setup_18.x`

This script prepares your system for Node.js 18 installation.

Pipe Symbol |

|

Takes output from left command and sends it to right command.

`sudo bash -`

Runs the downloaded setup script using bash as administrator.

Simple Meaning

This command configures your Linux system so Node.js 18 can be installed properly.

3. Install Node.js and Git

Bash

```
sudo dnf install -y nodejs git
```

What it does

Installs:

- Node.js
- Git

using the DNF package manager.

Breaking it Down

install

Tells DNF to install packages.

nodejs

Installs:

- Node runtime
- npm (Node Package Manager)

Used for running JavaScript backend apps like:

- React
 - Express
 - Next.js
-

git

Installs Git version control system.

Used for:

- Cloning repositories
 - Pushing code to GitHub
 - Managing projects
-

-y

Automatically confirms installation.

Simple Meaning

This command installs:

- Node.js for running JavaScript applications
 - Git for source code management
-

Real Workflow

Usually these commands are run together:

Bash

```
sudo dnf update -y
curl -fsSL https://rpm.nodesource.com/setup_18.x | sudo bash -
sudo dnf install -y nodejs git
```

After Installation

You can verify installation using:

Bash

```
node --v
npm --v
git --version
```

Viva Answers

What is DNF?

DNF is a package manager used in modern Linux distributions to install, update, and manage software packages.

What is Node.js?

Node.js is a JavaScript runtime environment used to run JavaScript outside the browser.

What is Git?

Git is a distributed version control system used to track and manage source code changes.

NOW TELL ME EVERYTHING ABOUT AWS AND SALESFORCE LIKE TAKING INTO REFERENCE MY ASSIGNMENTS , WHAT ALL EXAMINERS CAN ASK ME, ITS USAGE , APPLICATIONS, WHAT ARE THEY EXACTLY, ADV, DIS EVERYTHING

AWS (Amazon Web Services)

What is AWS?

Amazon Web Services is a **cloud computing platform** provided by Amazon.

Instead of buying physical servers and computers, companies rent:

- Servers
- Storage
- Databases
- Networking
- Security
- AI services

through the internet.

Simple Definition

AWS provides online computing resources like servers, storage, and databases on rent.

Real-Life Analogy

Without AWS:

Company buys:

- Physical servers
- Cooling systems
- Networking devices
- Maintenance staff

With AWS:

Company rents everything online from AWS

Why AWS is Used

Companies use AWS because:

- Cheap initially
- Scalable
- Fast deployment
- Highly secure
- No hardware maintenance

Main AWS Services

1. EC2 (Elastic Compute Cloud)

What is it?

Virtual computer/server in cloud.

You used this in your assignment.

Your Practical

You:

- Created EC2 instances
- Connected using SSH
- Installed NGINX
- Hosted website
- Transferred files between instances

Commands You Used

Bash

```
ssh -i key.pem ec2-user@ip
```

Used to connect to EC2.

Examiner Can Ask

Q1. What is EC2?

EC2 is a virtual server provided by AWS used to run applications in cloud.

Q2. Why use EC2?

- No physical hardware required
 - Scalable
 - Remote access
 - Pay-as-you-go
-

Q3. Difference between Public IP and Private IP?

Public IP

- Accessible from internet

Private IP

- Used inside same VPC/network
-

Q4. Why use PEM key?

PEM key is used for secure authentication while connecting to EC2 using SSH.

2. S3 (Simple Storage Service)

What is it?

Cloud storage service.

Used for:

- Images
 - Videos
 - Backups
 - Files
-

3. IAM (Identity and Access Management)

What is it?

Controls:

- Users
 - Roles
 - Permissions
-

Example

Admin can:

- Create users
 - Give limited access
-

4. VPC (Virtual Private Cloud)

What is it?

Private network inside AWS cloud.

Your EC2 instances communicate inside VPC.

Advantages of AWS

1. Scalability

Increase or decrease resources anytime.

2. Pay-as-you-go

Pay only for what you use.

3. High Availability

AWS servers run globally.

4. Security

Provides:

- Encryption
 - IAM
 - Firewalls
-

5. Fast Deployment

Server can be created within minutes.

Disadvantages of AWS

1. Costly if unmanaged

Wrong configurations increase bills.

2. Internet dependency

Cloud services need internet.

3. Learning curve

Many services make it complex initially.

Applications of AWS

Used in:

- Netflix
 - Amazon
 - Startups
 - Banking
 - AI systems
 - Hosting websites
 - Mobile apps
 - Big data
-

NGINX in Your AWS Assignment

Why NGINX?

You installed:

Bash

```
sudo yum install nginx -y
```

because EC2 alone is just a server.

NGINX converts it into a web server.

Examiner Questions from Your Assignment

Q. Why did you use NGINX?

To host and serve static webpages from EC2 instance.

Q. What is SSH?

SSH is a secure protocol used for remote login into servers.

Q. What is SCP?

SCP is Secure Copy Protocol used to transfer files securely between systems.

Q. Why chmod 400 key.pem?

SSH requires private keys to have secure permissions.

SALESFORCE

What is Salesforce?

Salesforce is a **cloud-based CRM platform**.

CRM = Customer Relationship Management.

Simple Definition

Salesforce is used to manage customers, sales, records, automation, and business processes online.

Real-Life Example

Suppose a company has:

- Customers
- Employees
- Products
- Orders

Salesforce stores and manages all data centrally.

Your Assignments in Salesforce

You created:

- Custom objects
 - Fields
 - Validation rules
 - Visualforce pages
 - Apex controllers
 - CRUD operations
-

What is a Custom Object?

Example:

```
Student__c  
Product__c  
Employee__c
```

These are database tables in Salesforce.

What are Fields?

Columns in object/table.

Example:

- Name
 - Roll Number
 - Phone Number
-

What is Apex?

Salesforce backend programming language.

Similar to Java.

Your Controller Example

Java

```
public class PRODUCTCONTROLLER
```

Handles:

- Insert
- Update
- Delete
- Edit

operations.

What is Visualforce?

Frontend page technology in Salesforce.

Used to create UI.

Example:

HTML

```
<apex:page>
```

CRUD Operations

Create

Insert new records.

Java
insert product;

Read

Fetch records.

Java
SELECT Name FROM Product__c

Update

Modify existing records.

Java
update product;

Delete

Remove records.

Java
delete product;

Validation Rules

Used to restrict invalid data.

Example:

- Phone number must be 10 digits
- Expiry date cannot be past date

Examiner Questions for Salesforce

Q1. What is Salesforce?

Salesforce is a cloud-based CRM platform used to manage customer and business data.

Q2. What is CRM?

CRM stands for Customer Relationship Management.

Q3. What is Apex?

Apex is Salesforce's backend programming language used for business logic.

Q4. What is Visualforce?

Visualforce is a framework used to create user interfaces in Salesforce.

Q5. What is a Custom Object?

Custom object is a user-created database table in Salesforce.

Q6. Difference between Standard and Custom Object?

Standard Object

Predefined by Salesforce.

Example:

- Account
- Contact

Custom Object

Created by user.

Example:

- Student__c
-

Q7. Why "__c" is used?

"__c" indicates custom objects and custom fields in Salesforce.

Q8. What is SOQL?

Salesforce Object Query Language.

Used to fetch data.

Example:

```
SQL  
SELECT Name FROM Student__c
```

Q9. Why use reRender in Visualforce?

```
HTML  
reRender="form"
```

Refreshes page components after operations.

Q10. What is Lightning?

Modern Salesforce UI framework.

Advantages of Salesforce

1. Cloud-Based

No installation required.

2. Easy Automation

Can automate workflows.

3. Secure

Role-based access.

4. Scalable

Can handle large business data.

5. Fast Development

Easy to create applications.

Disadvantages of Salesforce

1. Expensive

Enterprise features cost more.

2. Internet Required

Works mainly online.

3. Customization Complexity

Advanced customizations require Apex knowledge.

Applications of Salesforce

Used in:

- Banking
 - Healthcare
 - Sales management
 - Customer support
 - HR systems
 - Inventory management
 - Student management
-

Difference Between AWS and Salesforce

AWS	Salesforce
Cloud infrastructure platform	CRM platform
Provides servers and storage	Manages customer/business data
Used for hosting apps	Used for business workflows
Example: EC2, S3	Example: Apex, Visualforce

Ultimate Viva Closing Answer

AWS is a cloud computing platform used to provide infrastructure services like servers and storage, while Salesforce is a cloud-based CRM platform used to manage customer and business applications with tools like Apex and Visualforce.

WHAT IS LIGHTNING WEB COMPONENT, DIFF BTN VALIDATIOON AND VERIFICATION,

What is Lightning Web Component (LWC)?

Definition

Lightning Web Component (LWC) is a modern frontend framework in Salesforce used to build:

- Fast UI
- Dynamic web pages
- Reusable components

using:

- HTML
- JavaScript
- CSS

Simple Understanding

LWC is like React/Angular but made for Salesforce.

It is used to create:

- Forms
- Buttons
- Dashboards
- Data tables
- Dynamic applications

inside Salesforce.

Example

Suppose you create:

- Student Form
- Employee Portal
- Product Management UI

using buttons, tables, and validations.

That can be built using LWC.

Why LWC is Used

Older Salesforce UI technology:

Visualforce
Aura Components

New modern technology:

Lightning Web Components

LWC is:

- Faster
 - Lightweight
 - Modern
 - Easy to maintain
-

Technologies Used in LWC

Each component mainly has:

1. HTML File

UI design.

Example:

HTML

```
<template>  
  <lightning-button label="Save"></lightning-button>  
</template>
```

2. JavaScript File

Logic handling.

Example:

JavaScript

```
handleClick() {  
    alert('Button Clicked');  
}
```

3. CSS File

Styling.

4. XML File

Configuration file.

Controls:

- Where component appears
 - App page visibility
-

Features of LWC

1. Reusable Components

Create once, use many times.

2. Fast Performance

Uses modern browser engine.

3. Easy Integration

Works with Apex and Salesforce database.

4. Secure

Uses Salesforce security model.

Examiner Questions on LWC

Q1. What is LWC?

LWC is a modern JavaScript-based framework in Salesforce used to build fast and reusable UI components.

Q2. Difference between Visualforce and LWC?

Visualforce	LWC
Older technology	Modern technology
Server-side rendering	Client-side rendering
Slower	Faster
HTML-like syntax	Pure HTML + JS

Q3. Which languages are used in LWC?

HTML, JavaScript, CSS, and XML.

Q4. Why is LWC faster?

Because it uses modern browser standards and client-side rendering.

VALIDATION vs VERIFICATION

This is a VERY common viva question.

1. Validation

Definition

Validation checks whether input data is:

- Correct
- Proper format
- Acceptable

BEFORE processing.

Simple Meaning

"Is the entered data valid?"

Examples

Phone Number Validation

Must contain 10 digits

Email Validation

Must contain @gmail.com

Salesforce Example

Validation Rule:

Expiry Date cannot be less than today

Types of Validation

- Client-side validation
 - Server-side validation
-

Why Validation is Important

Prevents:

- Wrong data
 - Empty fields
 - Invalid records
-

Examiner Viva Answers

Q. What is Validation?

Validation is the process of checking whether entered data follows required rules and formats.

2. Verification

Definition

Verification checks whether:

- Data
- Software
- System

matches expected requirements/specifications.

Simple Meaning

“Did we build the system correctly?”

Examples

OTP Verification

Checks whether OTP entered is correct.

Email Verification

Checks whether email actually belongs to user.

Software Verification

Checking whether application works according to requirements.

Validation vs Verification

Validation	Verification
Checks correctness of input	Checks correctness of system
Prevents invalid data	Ensures system meets requirements
Done before processing	Done during/after development
Example: phone number check	Example: OTP match

Easy Real-Life Example

Validation

College form says:

Roll number must be numeric

System checks format.

Verification

College sends OTP/email confirmation.

System checks if user is real.

In Salesforce Assignment

Validation Example

Phone number must be 10 digits

using Validation Rule.

Verification Example

After saving data:

- Check record inserted correctly
 - Check update worked properly
-

Very Important Viva Question

Q. Difference between Validation and Verification?

Validation checks whether input data is correct, while verification checks whether the system or software works according to requirements.